Simple null simulation for the FPR convergence rate

J. Dedecker, M.-L. Taupin, V. Runge and A.-S. Tocquet

2026-07-01

Contents

1 Objective **1**

2 Parameters **2**

3 Original model-free screening test **2**

4 One nested null replication **3**

5 Monte Carlo FPR estimates **4**

6 FPR convergence-rate summary **4**

7 Log-log rate estimate **5**

8 Log-log plot **6**

9 Interpretation checklist **7**

10 Save results **8**

1 Objective

This simulation studies the null false-positive rate of the original model-free screening test in the simplest possible setting.

For each replication, generate one Gaussian predictor column X and one independent Gaussian response Y:

$$X_i \sim \mathcal{N}(0,1), \qquad Y_i \sim \mathcal{N}(0,1), \qquad X_i \perp Y_i.$$

There is only one null covariate per replication. Repeating the experiment many times is equivalent to studying many independent Gaussian null columns.

For each replication, one pair (X, Y) is generated at the largest sample size $n_{\max}$. The results for smaller sample sizes use the first n observations. Thus each replication uses the **same nested design** for every sample size.

Under correct FPR control, the rejection probability tends to the nominal level q. The quantity whose convergence can have rate $C/\sqrt{n}$ is

$$|\mathrm{FPR}_n - q|\,,$$

not FPR_n itself. On a log-log plot, the reference slope for this deviation is $-1/2$.

2 Parameters

```
knitr::opts_chunk$set(
  echo = TRUE,
  eval = TRUE,
  cache.path = "simple_fpr_rate_cache/"
)

library(parallel)
library(knitr)

# Nominal two-sided false-positive rate.
q <- 0.5

# Nested sample sizes. Each must be at most max(n_values).
n_values <- round(1.5^(5:10)*500)
n_max <- max(n_values)

# Number of independent null columns / replications.
# Use 500 for a quick check; use 5000 or more for a final study.
B <- 10000L

# Reproducible, distinct seeds for the B replications.
base_seed <- 20260701L
replication_seeds <- base_seed + seq_len(B)

# Parallel settings.
n_cores <- min(8L, parallel::detectCores(logical = FALSE))
```

3 Original model-free screening test

```
screening_test_ML <- function(X, Y, q_n = 0.10) {
  X <- as.matrix(X)
  Y <- as.numeric(Y)

  n <- nrow(X)

  if (ncol(X) != 1L) {
    stop("This simple study expects exactly one predictor column.")
  }
  if (length(Y) != n) {
```

```
    stop("Y must have the same number of observations as rows of X.")
  }

  x_mean <- mean(X[, 1L])
  y_mean <- mean(Y)

  x_centered <- X[, 1L] - x_mean
  y_centered <- Y - y_mean

  sxx <- sum(x_centered^2)
  if (sxx <= 0) {
    stop("X has zero sample variance.")
  }

  tau_hat <- sum(x_centered * y_centered) / sxx
  b_hat <- y_mean - tau_hat * x_mean

  residuals_hat <- Y - b_hat - tau_hat * X[, 1L]

  # White variance estimator from the model-free screening procedure.
  var_x_n <- sxx / n
  V_hat <- mean(x_centered^2 * residuals_hat^2) / var_x_n^2

  gamma_hat <- sqrt(n) * tau_hat / sqrt(V_hat)
  threshold_gamma <- qnorm(1 - q_n / 2)

  list(
    score = gamma_hat,
    selected = abs(gamma_hat) >= threshold_gamma,
    threshold = threshold_gamma
  )
}
```

4 One nested null replication

One X vector and one independent Y vector are generated at length $n_{\max}$. For every n, the test uses the prefix $(X_1, \ldots, X_n, Y_1, \ldots, Y_n)$.

```
run_one_null_replication <- function(seed, n_values, q) {
  set.seed(seed)

  n_max <- max(n_values)

  # One Gaussian null column and one independent response.
  X_full <- rnorm(n_max)
  Y_full <- rnorm(n_max)

  selected <- logical(length(n_values))

  for (k in seq_along(n_values)) {
    n <- n_values[k]
```

```
    test <- screening_test_ML(
      X = matrix(X_full[seq_len(n)], ncol = 1L),
      Y = Y_full[seq_len(n)],
      q_n = q
    )

    selected[k] <- test$selected
  }

  selected
}
```

5 Monte Carlo FPR estimates

```
one_draw <- function(seed) {
  run_one_null_replication(
    seed = seed,
    n_values = n_values,
    q = q
  )
}

replication_results <- if (.Platform$OS.type == "windows" || n_cores <= 1L) {
  lapply(replication_seeds, one_draw)
} else {
  parallel::mclapply(
    X = replication_seeds,
    FUN = one_draw,
    mc.cores = n_cores,
    mc.set.seed = FALSE,
    mc.preschedule = FALSE
  )
}

# Rows are independent Gaussian null columns; columns are sample sizes.
selection_matrix <- do.call(rbind, replication_results)
colnames(selection_matrix) <- paste0("n=", n_values)

# FPR estimate for each n.
fpr_hat <- colMeans(selection_matrix)
```

6 FPR convergence-rate summary

The Monte Carlo standard error below measures uncertainty from the finite number B of independent null columns. A clean rate assessment requires the estimated deviation to be appreciably larger than this Monte Carlo noise.

```r
mc_se <- sqrt(fpr_hat * (1 - fpr_hat) / B)
fpr_error <- fpr_hat - q
abs_fpr_error <- abs(fpr_error)
scaled_error <- sqrt(n_values) * abs_fpr_error

fpr_rate_summary <- data.frame(
  n = n_values,
  FPR_hat = fpr_hat,
  FPR_minus_q = fpr_error,
  abs_FPR_minus_q = abs_fpr_error,
  MC_SE = mc_se,
  sqrt_n_times_abs_FPR_minus_q = scaled_error,
  signal_to_MC_noise = abs_fpr_error / mc_se
)

knitr::kable(
  fpr_rate_summary,
  digits = 6,
  caption = paste0(
    "Null FPR study with B = ", B,
    " independent Gaussian columns and q = ", q
  )
)
```

Table 1: Null FPR study with B = 10000 independent Gaussian columns and q = 0.5

	n	FPR_hat	FPR_minus_q	abs_FPR_minus_q	MC_SE	sqrt_n_times_abs_FPR_minus_q	signal_to_MC_noise
n=3797	3797	0.4934	-0.0066	0.0066	0.005000	0.406691	1.320115
n=5695	5695	0.4975	-0.0025	0.0025	0.005000	0.188663	0.500006
n=8543	8543	0.5093	0.0093	0.0093	0.004999	0.859584	1.860322
n=12814	12814	0.5015	0.0015	0.0015	0.005000	0.169798	0.300001
n=19222	19222	0.4959	-0.0041	0.0041	0.005000	0.568438	0.820028
n=28833	28833	0.4985	-0.0015	0.0015	0.005000	0.254704	0.300001

7 Log-log rate estimate

The slope is estimated by regressing $\log|\widehat{\mathrm{FPR}}_n - q|$ on $\log n$.

Only sample sizes with a deviation larger than two Monte Carlo standard errors are used in the primary slope fit. This prevents a random estimate very close to q from dominating the log-log regression.

```r
usable <- abs_fpr_error > 2 * mc_se

if (sum(usable) >= 3L) {
  rate_fit <- lm(
    log(abs_fpr_error[usable]) ~ log(n_values[usable])
  )

  estimated_slope <- unname(coef(rate_fit)[2L])
  rate_fit_summary <- data.frame(
```

```
    n_points_used = sum(usable),
    estimated_log_log_slope = estimated_slope,
    target_slope = -0.5
  )

  knitr::kable(
    rate_fit_summary,
    digits = 4,
    caption = "Log-log slope estimate for the FPR deviation"
  )
} else {
  estimated_slope <- NA_real_
  message(
    "Too few deviations exceed two Monte Carlo standard errors. ",
    "Increase B before interpreting a log-log slope."
  )
}
```

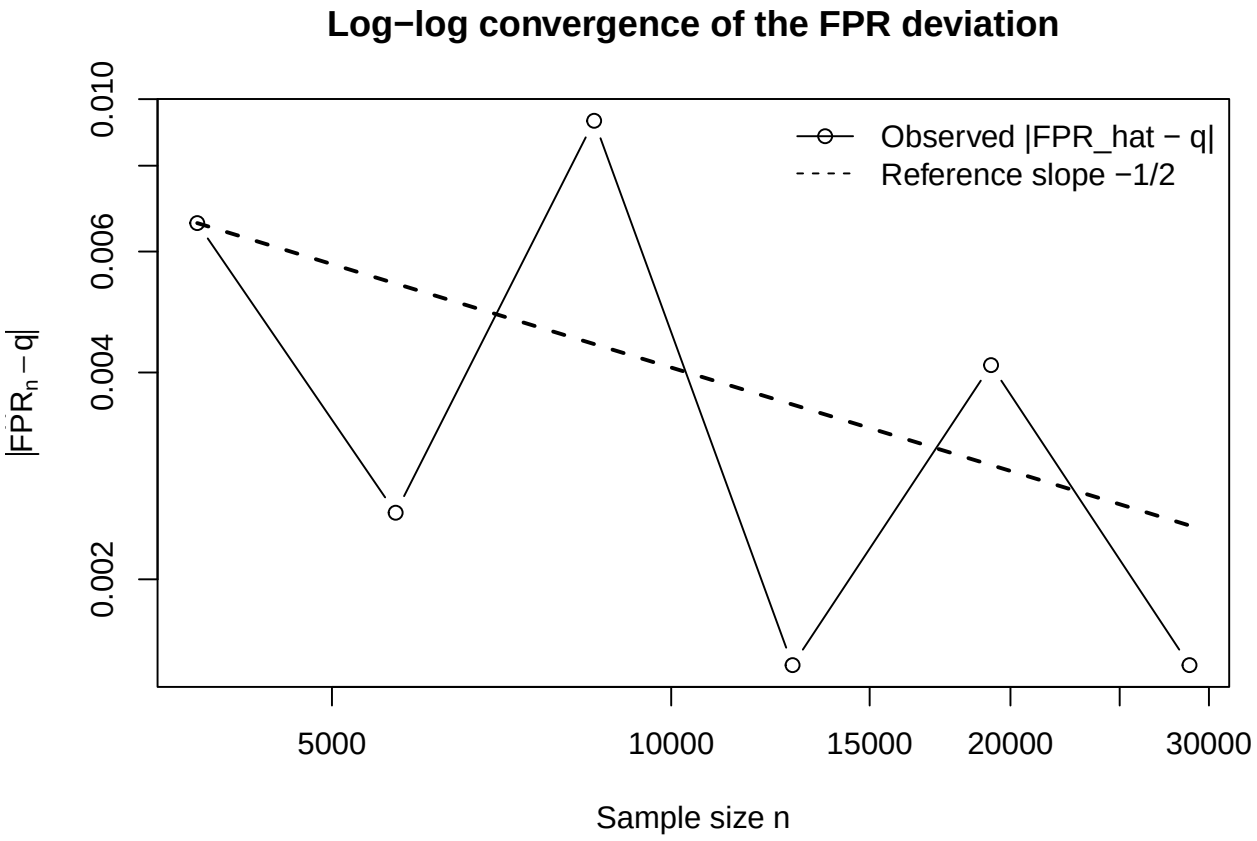
```
## Too few deviations exceed two Monte Carlo standard errors. Increase B before interpreting a log-log s
```

8 Log-log plot

The solid line is the estimated absolute FPR deviation. The dashed line is a reference line with slope $-1/2$, anchored at the first positive observed deviation.

```
positive <- abs_fpr_error > 0

plot(
  n_values[positive],
  abs_fpr_error[positive],
  log = "xy",
  type = "b",
  xlab = "Sample size n",
  ylab = expression(abs(hat(FPR)[n] - q)),
  main = "Log-log convergence of the FPR deviation"
)

if (any(positive)) {
  n_anchor <- n_values[which(positive)[1L]]
  error_anchor <- abs_fpr_error[which(positive)[1L]]

  reference_n <- range(n_values)
  reference_error <- error_anchor * (reference_n / n_anchor)^(-0.5)

  lines(
    reference_n,
    reference_error,
    lty = 2,
    lwd = 2
  )

  legend(
```

```
    "topright",
    legend = c(
      "Observed |FPR_hat - q|",
      "Reference slope -1/2"
    ),
    lty = c(1, 2),
    pch = c(1, NA),
    bty = "n"
  )
}
```



9 Interpretation checklist

- The estimated FPR should remain close to `q = 0.10`.
- The column `sqrt_n_times_abs_FPR_minus_q` should be roughly stable when a $C/\sqrt{n}$ approximation is appropriate.
- The estimated log-log slope should be near -0.5, but only when the deviations exceed their Monte Carlo uncertainty.
- A slope far from -0.5 can result from finite-sample bias, insufficient `B`, or a rate faster than $1/\sqrt{n}$, in which case Monte Carlo noise masks the true deviation.

10 Save results

```
saveRDS(
  list(
    parameters = list(q = q, n_values = n_values, B = B),
    fpr_rate_summary = fpr_rate_summary,
    selection_matrix = selection_matrix
  ),
  file = "simple_null_fpr_convergence_results.rds"
)

write.csv(
  fpr_rate_summary,
  file = "simple_null_fpr_convergence_summary.csv",
  row.names = FALSE
)
```